Lec-31
24/07/26 7:10 PM

Given an array of integers, for each element in the array, find the **next greater element** (NGE) in the array. The next greater element for an element *x* is the first element *y* to the right of *x* in the array such that *y* > *x*. If no such element exists, the NGE for that element is **-1**.

0 1 2 3 4 5 6 7 8
[50, 5, 3, 2, 8, 7, 9, 11, 3]
-1 8 8 8 9 9 11 -1 -1

NGE PGE
NSE PSE

[50, 5, 3, 2, 8, 7, 9, 11, 3]

for(i=0 i<n i++){
while(!st.isEmpty() && arr[i] > arr[st.peek()]){
ans[st.pop()] = arr[i]
}
st.push(i)
}

while(!st.isEmpty()){
ans[st.pop()]=-1

```java
int[] arr = { 4, 5, 2, 10 };
```

```java
public static void NGE(int[] arr) {
        Stack<Integer> st = new Stack<>();
        int[] ans = new int[arr.length];
        for (int i = 0; i < arr.length; i++) {
            while (!st.isEmpty() && arr[i] > arr[st.peek()]) {
                ans[st.pop()] = arr[i];
            }
            st.push(i);
        }
        while (!st.isEmpty()) {
            ans[st.pop()] = -1;
        }
        for (int i = 0; i < ans.length; i++) {
            System.out.println(arr[i] + " " + ans[i]);
        }
    }
```

10 > 2

Example 1

Input

5
30
35
40
38
35

Output

1 2 3 1 1 END

5
30 35 40 38 35
1 2 3 1 1

[50, 5, 3, 2, 8, 7, 9, 11, 3]
1 1 1 1 4 1 6 7 1
0 1 2 3 4 5 6 7 8

O(n)

Ptr 132

for(i=0 i<n i++){
while(!st.isEmpty() && arr[i] >= arr[st.peek()]){
st.pop()
}
//span cal
if(st.isEmpty()){
ans[i] = i+1
}
else{
ans[i] = i-st.peek()
}
st.push(i)
}

0 1 2 3 4 5 6
[2, 3, 5, 4, 6, 1, 7]

2x5=10
3x4=12
5x1=5
4x3=12
6x1=6
1x7=7
7x1=7

[2, 3, 5, 4, 6, 1, 7, 0]

NSE

h x (R-L-1)
5 x (R-L-1)
5 x (3-1-1) = 5
6 x (5-3-1) = 6
4 x (5-1-1) = 12
3 x (5-0-1) = 12
2 x (5) = 10

19-1-1 = 6

R = arr.length

7 x (7-5-1) = 7
1 x 7 = 7

[2, 3, 5, 4, 6, 1, 7]
0 1 2 3 4 5 6

5x(3-1-1)=5 R=5

for(i=0 i<n i++){
while(!st.isEmpty() && arr[i] < arr[st.peek()]){
R=i, h=arr[st.pop()]
if(st.isEmpty()){
area = max(area, h x R)
}
else{ L=st.peek()
area = max(area, h x (R-L-1))
}
}
st.push(i)
}

6x(5-3-1)
4x(5-1-1)=12
3x(5-0-1)=12
2x5=10

R=arr.length

[2, 3, 5, 4, 6, 1, 7, 0]

```java
int r=arr.length;
        while(!st.isEmpty()) {
            int h=arr[st.pop()];
            if(st.isEmpty()) {
                area=Math.max(area, h*r);
            }
            else {
                int l=st.peek();
                area=Math.max(area, h*(r-l-1));
            }
        }
```

7x(7-5-1)=7

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

[1, 0, 1, 0, 0] → 1
[2, 1, 1, 1, 1] → 5
[3, 0, 2, 3, 2] → 6
[4, 0, 3, 0, 0] → 4